GOODFOOD

Restaurant Survey & Table-Side Ordering System

Project Documentation

Project Planning & Management | Literature Review | Requirements Gathering

Graduation Project

1. Project Overview

1.1 Project Idea

GoodFood is a digital system designed for one restaurant. It combines table-side ordering, kitchen order management, meal-level reviews, service ratings, customer suggestions, menu administration, and restaurant analytics. Customers enter their name, table number, and party size, browse the menu, confirm an order, and submit feedback. Chefs process orders and view meal feedback only. Administrators manage menu items and chef accounts, monitor orders, and review all feedback and performance indicators.

1.2 Problem Statement

Traditional ordering can cause delays while customers wait for staff, mistakes when orders are communicated verbally, and limited structured feedback about individual meals and service quality. GoodFood creates a shared digital workflow from customer ordering to kitchen processing and feedback analysis.

1.3 Objectives

- Allow customers to browse the digital menu and place orders directly from their table.
- Reduce avoidable order-taking delays and order transcription mistakes.
- Show order progress with clear statuses such as Pending, Preparing, and Ready.
- Collect separate ratings for individual meals and overall service.
- Collect customer suggestions and comments.
- Help administrators identify highly rated, average-rated, and poorly rated meals.
- Provide secure role-based access and controlled chef-account management.
- Report useful operational indicators such as average preparation time when timestamps are available.

1.4 Scope

Included

- One restaurant; customer, chef, and admin roles.
- Customer details entry, menu browsing, meal details, search/filter, cart, checkout confirmation, order confirmation, order tracking, and feedback.
- Chef login, kitchen order queue, order-status updates, and meal-feedback view.
- Admin login, menu management, order search/filter, Chef Management, all-feedback view, analytics, and reports.
- Firebase Authentication and Cloud Firestore, with trusted backend logic for privileged actions.

Excluded / Constraints

- Real payment gateway integration is excluded; checkout confirms the order only.
- Customer sign-up, login, and profile management are not required.
- Multi-restaurant tenancy, delivery-driver management, table reservations, payroll, and full accounting are outside the core scope.

2. Project Planning & Management

2.1 Development Approach

The team will work iteratively: finalize requirements, design screen flows and data contracts, implement modules in parallel, integrate them, test end-to-end scenarios and role permissions, fix issues, then prepare the final demo. Shared Firestore fields, role rules, and order statuses should be agreed before implementation diverges.

2.2 Team Responsibilities

Responsibilities are listed here at the start of the planning section—not on the cover page.

Team Member	Assigned Responsibilities	Tasks
Mohamed Ibrahim	Orders + Chef System + Chef Management	• Display customer orders and order details. • Track order status (Order Tracking). • Chef Dashboard to display incoming orders. • Chef updates order status (Preparing / Ready). • Admin Orders page to monitor, search, and filter orders.
Omar Hesham	Auth + Cart + Checkout	• Splash Screen and Device Setup. • Admin and Chef login. • Manage item quantities in the Cart. • Checkout confirmation. • Create the order and display Order Confirmation. • Admin & Chef Profile Pages.
Mohamed Mahmoud	Menu Management System	• Customer Menu to display meals. • Meal Details. • Search and filter meals by category. • Admin Menu Management. • Add, edit, and delete meals, and set meal availability.
Ibrahim Mohamed	Feedback + Analytics + chef Management	• Rate individual meals and service. • Collect customer suggestions and comments. • Admin Dashboard and statistics. • Chef Meal Feedback. • Reports and restaurant performance analysis. • Add, edit, deactivate, reactivate and delete chefs

2.3 Timeline and Milestones

The project completion deadline is 22 November 2026, one week before the submission deadline on 29 November 2026. The final week is reserved for regression checks, documentation and presentation packaging, and submission readiness. The discussion is scheduled for 4 December 2026.

Dates	Planned activities	Milestone / deliverable	Owner(s)
Oct 16–18	Confirm project scope, screen list, Firebase data contracts, and order statuses.	M1 — Scope and data contracts finalized.	All team members
Oct 19–25	Complete core UI and navigation, staff authentication/device setup, and menu foundation.	M2 — Main navigation, staff login, and menu flow ready.	Omar; Mohamed Mahmoud
Oct 26–Nov 1	Implement cart, checkout confirmation, order creation, customer order tracking, and chef order queue.	M3 — Customer orders appear in the chef queue.	Omar; Mohamed El Amir
Nov 2–8	Complete chef order processing, status updates, admin order search/filter, Chef Management, feedback, and analytics.	M4 — Core features integrated.	All team members
Nov 9–15	Integrate all modules, run	M5 — Integrated build tested.	All team members

	end-to-end tests, fix critical bugs, and verify role permissions.		
Nov 16–22	Finish all features, resolve remaining issues, finalize documentation, and freeze the project.	M6 — Project completed by Nov 22.	All team members
Nov 23–26	Run final regression checks, validate the demo, and prepare screenshots and presentation materials.	M7 — Submission package ready.	All team members
Nov 27–28	Final review, backup build/files, and submission readiness check; no new features.	M8 — Ready to submit.	All team members
Nov 29	Project submission deadline.	Submission.	All team members
Nov 30–Dec 3	Prepare for the discussion, rehearse the demo, and make only essential fixes if needed.	M9 — Ready for discussion.	All team members
Dec 4	Project discussion.	Final discussion.	All team members

2.4 Milestone Acceptance Criteria

- M1: Scope, exclusions, stakeholders, and priorities are agreed.
- M2: Screen flows, data fields, role permissions, and order statuses are documented.
- M3: Staff-only screens are protected and customers can browse the menu.
- M4: Customer can submit an order, receive confirmation, and the chef can see it.
- M5: Chef can update order status; admin can search orders and securely manage chef accounts.
- M6: Meal ratings, separate service ratings, suggestions, and role-limited feedback views work.
- M7: Unauthorized actions are denied and critical end-to-end tests pass.
- M8: Stable demo build, final documentation, and presentation are ready.

2.5 Risks and Mitigation

Risk	Impact	Mitigation
Unauthorized staff account creation	High	No public staff sign-up; provision first admin through a controlled process; create chefs only through backend logic that verifies admin privileges.
Permissions enforced only in UI	High	Enforce authorization in Firestore Security Rules and trusted backend functions; test all roles.
Duplicate or inconsistent orders	High	Use stable order IDs, validation, server timestamps, defined status transitions, and duplicate-submit protection where feasible.
Conflicting data fields across modules	Medium/high	Agree on collection names, required fields, and status enums before integration.
Connectivity interruptions	Medium	Show loading/error states, allow safe retries, and do not display success until the order is saved.
Scope creep	Medium	Prioritize the end-to-end order flow and feedback requirements; keep real payment and multi-restaurant support out of scope.

2.6 Success Metrics (KPIs)

Metric	Evaluation method	Proposed target
Order submission reliability	Successful submissions / valid checkout attempts in controlled tests.	At least 95%

Status update visibility	Time between chef update and authorized views receiving the update.	A few seconds under normal connectivity
End-to-end order workflow	Test from customer checkout through chef Ready status.	All critical test cases pass
Role-based access	Attempt restricted actions using each role.	All tested unauthorized actions denied
Feedback correctness	Compare stored ratings and summaries with a test dataset.	All checked calculations correct
Usability	Representative users complete key tasks without help.	At least 80% task completion in a small test

Targets are proposed acceptance criteria, not measured production results.

3. Literature Review

This review compares GoodFood with established restaurant technology solutions using their publicly described features. The comparison is based on published documentation, not hands-on testing by the team.

3.1 Toast Restaurant Platform and Online Ordering

Toast describes a restaurant platform connecting order taking, online ordering, and kitchen operations. Its online-ordering documentation explains how menus are configured and how orders can be sent to the kitchen. This supports GoodFood's design of connecting customer ordering to the chef workflow.

Strengths: integrated order-to-kitchen workflow, menu configuration, and operational tools. Limitations for this comparison: it is a broad commercial platform, while GoodFood is a focused student prototype; commercial payment and business features are outside GoodFood's scope.

3.2 Square Online Ordering

Square describes online ordering linked to its point-of-sale ecosystem, with menu and order workflows connected to restaurant operations. Its menu-management documentation describes controlling menu availability and visibility across channels.

Strengths: unified menu/order management and less manual re-entry. Limitations for this comparison: its published workflows include online ordering and POS integrations, while GoodFood focuses on dine-in table-side ordering, meal-specific ratings, and role-limited feedback.

3.3 Kitchen Display Systems

Kitchen display systems make incoming orders and their preparation progress visible to kitchen staff. This supports GoodFood's dedicated chef dashboard and explicit Preparing/Ready statuses. A kitchen display alone, however, does not cover the project's separate service rating, customer suggestions, and analytics workflow.

3.4 Comparison with GoodFood

Capability	Toast / Square-type solutions	GoodFood
Digital menu and ordering	Available in commercial ordering ecosystems.	Table-side menu on restaurant tablet/mobile.
Kitchen order workflow	Order routing and kitchen operations supported.	Chef dashboard and defined order statuses.
Menu management	Commercial menu management tools.	Admin adds/edits/removes meals and changes availability.
Meal-specific feedback	Not established as a core feature by the cited pages.	Core requirement.
Separate service rating and suggestions	Not established as a core feature by the cited pages.	Core requirement.
Chef sees meal feedback only	Not established as a specific role rule by the cited pages.	Explicit role-based requirement.

Real payment	Available in commercial ecosystems.	Excluded; checkout confirms the order only.
Target scope	Commercial restaurant platform.	Single-restaurant academic prototype.

3.5 Research Gap and Contribution

The reviewed solutions show the value of connecting menu, order intake, and kitchen operations. GoodFood focuses on combining dine-in table-side ordering with structured meal ratings, a separate service rating, suggestions, and role-based feedback access. The chef can use meal feedback to improve food preparation, while the administrator can review the wider service experience and performance indicators.

3.6 References

1. Toast. (n.d.). Restaurant POS. https://pos.toasttab.com/restaurant-pos/
2. Toast Support. (n.d.). Getting started with online ordering. https://support.toasttab.com/en/article/Getting-Started-Online-Ordering
3. Toast Developer Documentation. (n.d.). Kitchen display system overview. https://dev.toasttab.com/doc/platformguide/platformKDSOverview.html
4. Square. (n.d.). Online Ordering System for Restaurants. https://squareup.com/us/en/online-ordering
5. Square Support. (n.d.). Manage your menus across locations and sales channels. https://squareup.com/help/us/en/article/8553-manage-your-menus-across-locations-and-sales-channels

4. Requirements Gathering

4.1 Stakeholders and Needs

Stakeholder	Needs
Customer	Browse/search menu, understand meal details and price, manage cart, confirm order, view order status, rate meals and service separately, and submit suggestions.
Chef	View incoming orders and item quantities, update order status, and view meal feedback only.
Administrator	Manage menu and availability, monitor/search orders, manage chef accounts, view all feedback, and review analytics/reports.
Restaurant owner / evaluator	A reliable prototype with clear security boundaries and demonstrable operational value.
Development team	Stable data contracts, clear module ownership, testable requirements, and integration practices.

4.2 Functional Requirements

ID	Requirement	Priority
FR-01	Customer enters name, table number, and party size without creating an account.	Must
FR-02	System displays menu items with name, description, price, category, image if available, and availability.	Must
FR-03	Customer can search and filter menu items by category.	Must
FR-04	Customer can add/remove items and change cart quantities.	Must
FR-05	System displays an order summary and requires confirmation before creating an order.	Must
FR-06	System creates an order and displays confirmation after successful submission.	Must
FR-07	Customer can see the current status of	Must

	their order.	
FR-08	Chef can securely log in and view incoming orders.	Must
FR-09	Chef can update order status using defined values such as Preparing and Ready.	Must
FR-10	Admin can search and filter restaurant orders.	Must
FR-11	Admin can add, edit, remove, and change availability of menu items.	Must
FR-12	Admin can create and deactivate chef accounts through an authorized backend workflow.	Must
FR-13	Customer can rate individual meals.	Must
FR-14	Customer can submit an overall service rating separately from meal ratings.	Must
FR-15	Customer can submit suggestions and comments.	Must
FR-16	Admin can view all feedback and analytics.	Must
FR-17	Chef can view meal feedback only, not service-level feedback or admin-only analytics.	Must
FR-18	System summarizes ratings and identifies highly, average, and poorly rated meals.	Should
FR-19	System calculates average preparation/order time from valid timestamps when available.	Should
FR-20	First-time device setup stores the configured device type/role locally.	Should
FR-21	Admin and chef screens require authentication and role authorization.	Must

4.3 Non-Functional Requirements

Category	Requirement
Security	Use Firebase Authentication for staff login; enforce permissions with Firestore Security Rules and trusted backend logic, not UI hiding alone.
Account safety	No public admin/chef sign-up. Provision the initial admin securely; allow chef creation only after backend verification of admin privileges.
Privacy	Collect only necessary data and restrict customer/feedback data to authorized roles.
Performance	Common screens should load promptly; status updates should appear within a few seconds under normal connectivity.
Reliability	Show loading, empty, success, and error states. Do not show success if an order was not saved.
Usability	Use consistent navigation, readable contrast, clear status labels, and mobile/tablet-friendly layouts.
Maintainability	Keep modules separated, document shared fields/enums, and review changes before integration.
Compatibility	Test on intended Flutter-supported mobile/tablet devices and validate web support if included in the release.
Accessibility	Use readable text, large touch targets, clear labels, and do not rely on color alone to convey status.

4.4 Roles and Permissions

Action	Customer	Chef	Admin
Browse menu	Allowed	Allowed if needed	Allowed
Create order	Allowed	No	No
View kitchen orders	Own order status	Allowed	Allowed
Update order status	No	Allowed	Oversight if implemented
Manage menu	No	No	Allowed

Manage chef accounts	No	No	Allowed through trusted backend
View meal feedback	Can submit	Allowed	Allowed
View service rating/all feedback	Can submit	No	Allowed
View analytics	No	No	Allowed

4.5 Key Use Cases

Use Case 1: Place an Order

Actor: Customer. Preconditions: menu is available and table details have been entered. Flow: browse/search menu → view meal details → add items to cart → adjust quantities → review order → confirm checkout → system validates and creates the order → confirmation appears and order is sent to the chef queue. If submission fails, show an error and allow a safe retry.

Use Case 2: Process an Order

Actor: Chef. Flow: log in → view kitchen queue → open order → set Preparing → complete preparation → set Ready. Authorized customer/admin views receive the updated status. Record relevant timestamps.

Use Case 3: Manage Chef Accounts

Actor: Admin. Flow: open Chef Management → enter chef details → trusted backend verifies admin privileges → backend creates Firebase Authentication account and Firestore profile → show result. Admin may edit permitted details or deactivate an account. Never place Firebase Admin SDK credentials in the Flutter app.

Use Case 4: Submit Feedback

Actor: Customer. Flow: rate each meal and the service separately → optionally enter suggestions → submit → store feedback linked to relevant meal/order → admin can view all feedback; chef can view meal feedback only.

4.6 Data and Business Rules

- Staff profiles reference Firebase Authentication UID; passwords are never stored in Firestore.
- Suggested collections: users, menuItems, orders, feedback, and deviceSettings.
- Orders should store stable IDs, table number, customer name, party size, item snapshots, status, creation timestamp, and relevant status timestamps.
- Meal rating and service rating are separate fields and are not combined.
- Define order statuses centrally and validate transitions.
- Show rating counts alongside averages to avoid misleading summaries.
- Restrict chef feedback access at the data layer where practical.
- Checkout is a simulated order-confirmation workflow; do not claim that a real payment was processed.

5. Acceptance Testing Checklist

- Customer enters table details without an account.
- Unavailable menu items cannot be ordered.
- Cart quantities and totals update correctly.
- Confirmed order is created once and appears in chef queue.
- Chef status updates are reflected in authorized views.
- Unauthenticated users cannot access admin/chef screens.
- Chef cannot create accounts or change their own role.
- Only an authorized admin can create/deactivate chef accounts.
- Chef cannot access service ratings or admin analytics.

- Admin can view meal/service ratings, suggestions, and summary analytics.
- Menu changes appear in customer menu.
- Network errors do not result in false success.
- Checkout is clearly represented as order confirmation, not real payment.

6. Conclusion

GoodFood is a focused single-restaurant prototype that connects customer ordering, kitchen operations, and feedback analysis. Its success depends on a complete end-to-end order workflow, consistent shared data contracts, backend-enforced role permissions, and tests proving that ratings and analytics are calculated and displayed correctly.